HKUST – Department of Computer Science and Engineering

COMP3711: Design and Analysis of Algorithms – Fall 2026

Practice Midterm B (new questions in the style of the 2016 to 2026 papers)

Time: 2 hours Closed book, one A4 cheat sheet No calculator

Instructions. Answer all questions. Algorithms may be given in pseudocode or plain language; define the input and output of every subroutine. You may use any algorithm from class, homework or tutorials as a black box. Correctness proofs are not required unless asked; running times must be justified briefly.

Question	1	2	3	4	5	Total
Points	10	10	12	8	20	60

1. Asymptotic analysis [10 points]

(a) [5 points] Fill the last column with "A", "B", or "U" as in the lecture examples.

Case	$T_A(n)$	$T_B(n)$	Faster
1	$\Theta(n (\log n)^2)$	$\Theta(n^{1.1})$	
2	$\Omega(n^2)$	$O(n^{1.99})$	
3	$\Theta(\log(n!))$	$\Theta(n \log n)$	
4	$O(2^n)$	$\Omega(n^{100})$	
5	$T_A(n) = 3T_A(n/2) + n$	$\Theta(n^{1.6})$	

(b) [5 points] Derive the exact asymptotic solution from first principles (no Master Theorem), assuming n is a power of 4. Give the form $\Theta(n^{\alpha} (\log n)^{\beta})$.

$$T(1) = 1; \quad T(n) = 2 \cdot T(n/4) + \sqrt{n} \text{ for } n > 1.$$

2. Randomization [10 points]

(a) [4 points] An array A[1..n] contains distinct numbers. Algorithm Lucky repeats the following until it succeeds: pick an index uniformly at random, then scan A once to check whether the picked element is among the largest n/10 elements. What is the expected number of scans, and the expected running time?

(b) [6 points] n keys are hashed independently and uniformly into a table with n slots (chaining). Let a key be **lonely** if no other key lands in its slot. Derive the exact expected number of lonely keys, and then the exact expected number of keys that share a slot with at least one other key.

3. Decision trees and lower bounds [12 points]

(a) [5 points] Draw a decision tree for an algorithm that finds the **maximum** of three distinct numbers A[1], A[2], A[3] using comparisons of the form "A[i] < A[j]?". Each leaf must be labelled with the index of the maximum. Use as few comparisons as possible on every path.

(b) [7 points] Two sorted arrays B[1..n] and C[1..n] of distinct numbers are to be merged into one sorted array of 2n numbers by a comparison-based algorithm. Prove that any such algorithm makes $\Omega(n)$ comparisons in the worst case. (Hint: count how many different outputs are possible, where an output is described by which positions of the merged array come from B.)

4. Sorting facts [8 points]

For each scenario, name the sorting algorithm from class with the best worst-case asymptotic running time and state that running time. No explanation needed.

(i) n integers in the range $[1, n^2]$.
(ii) n records that must be sorted **stably** in O(n log n) worst-case time.
(iii) n numbers where each element is at most 3 positions away from its sorted position.
(iv) n numbers with the requirement of O(1) extra space and O(n log n) worst-case time.

5. Divide and conquer and heaps [20 points]

(a) [6 points] You are given k sorted arrays, each with m elements (n = km in total). Design a divide and conquer algorithm that merges them into one sorted array, and analyse its running time as a function of n and k.

(b) [6 points] Design an alternative algorithm for (a) based on a min-heap, and analyse its running time and extra space.

(c) [8 points] An array A[1..n] is **d-disordered**: every element is at most d positions away from its position in sorted order. Design an O(n log d) algorithm that sorts A. Explain why it is correct and derive the running time. (Hint: use a heap of size d + 1.)

Practice Midterm B – Solutions

1. Asymptotic analysis

(a) 1: **A**, $(\log n)^2$ grows slower than $n^{0.1}$. 2: **B**, A is at least n^2 and B is at most $n^{1.99}$, so B is always smaller. 3: **U**, $\log(n!) = \Theta(n \log n)$, so they are the same class and neither is "always faster". 4: **U**, A could be constant (then A) or 2^n (then B). 5: **A**, $T_A = \Theta(n^{\log_2 3}) = \Theta(n^{1.585}) < n^{1.6}$.

(b) $T(n) = 2T(n/4) + n^{1/2} = 2[2T(n/16) + (n/4)^{1/2}] + n^{1/2} = 4T(n/16) + 2\cdot(n^{1/2}/2) + n^{1/2} = 4T(n/16) + 2n^{1/2}$.
Level i has 2^i nodes of size $n/4^i$, each costing $(n/4^i)^{1/2} = n^{1/2}/2^i$, so every level costs exactly $n^{1/2}$.
General term: $T(n) = 2^k\, T(n/4^k) + k\, n^{1/2}$. Stop at $k = \log_4 n$: leaves $2^{\log_4 n} = n^{\log_4 2} = n^{1/2}$.
$T(n) = n^{1/2} + n^{1/2} \log_4 n = \mathbf{\Theta(\sqrt{n} \log n)}$ ($\alpha = 1/2$, $\beta = 1$).

2. Randomization

(a) Each trial succeeds with probability $p = (n/10)/n = 1/10$, independently. The number of trials is geometric with $E = 1/p =$ **10 scans**. Each scan is O(n), so the expected running time is $O(10n) =$ **O(n)**. (Checking "among the largest n/10" by one scan: count how many elements exceed the picked one; it is in the top tenth iff that count is less than n/10.)

(b) Let $L_i = 1$ if key i is lonely. The other $n - 1$ keys each avoid key i's slot with probability $1 - 1/n$, so $\Pr[L_i = 1] = (1 - 1/n)^{n-1}$. By linearity E[lonely] = $\mathbf{n(1 - 1/n)^{n-1}}$. Keys that share: n minus that, $\mathbf{n - n(1 - 1/n)^{n-1}}$ (about $(1 - 1/e)n \approx 0.63n$). Note the exponent is $n - 1$, not n, because the key itself is excluded; this is the detail graders look for.

3. Decision trees and lower bounds

(a) Root: A[1] < A[2]? If yes, compare A[2] < A[3]?: yes → leaf 3, no → leaf 2. If no, compare A[1] < A[3]?: yes → leaf 3, no → leaf 1. Every path has exactly 2 comparisons, which is optimal since $n - 1 = 2$ comparisons are needed to find a maximum of 3.

(b) The merged array has 2n positions, and the output is determined by which n of them hold elements of B (the relative order within B and within C is fixed). Any choice of n positions is achievable by some input, so there are C(2n, n) distinct outputs. A comparison algorithm is a binary decision tree whose leaves must cover all outputs, so its height is at least $\log_2 C(2n, n)$. Since $C(2n, n) \geq 2^n$ (C(2n, n) is the largest of the $2n + 1$ binomial coefficients that sum to 4^n, so it is at least $4^n/(2n + 1) \geq 2^n$ for $n \geq 1$), the height is at least n. Hence $\Omega(n)$ comparisons in the worst case. (The standard merge uses at most $2n - 1$, so this is tight up to a constant.)

4. Sorting facts

(i) Radix sort with base n: two digits, $\Theta(2(n + n)) = \mathbf{\Theta(n)}$. (Counting sort directly would cost $\Theta(n + n^2)$.) (ii) **Mergesort**, $\Theta(n \log n)$. (iii) **Insertion sort**: at most 3n inversions, $\Theta(n + 3n) = \mathbf{\Theta(n)}$. (iv) **Heapsort**, $\Theta(n \log n)$ in place.

5. Divide and conquer and heaps

(a) MergeAll(arrays 1..k): if k = 1 return that array; otherwise recursively merge the first k/2 arrays and the last k/2 arrays, then merge the two results with the standard linear merge. At each level of the recursion the total length of all arrays being merged is n, so each level costs O(n); there are $\log_2 k$ levels. Recurrence $T(k) = 2T(k/2) + O(n)$ with $T(1) = O(1)$, solution **O(n log k)**.

(b) Put the first element of each array into a min-heap as (value, array id, index). Repeat n times: extract the minimum, append it to the output, and insert the next element of the same array if any. The heap never exceeds k entries, so every operation is O(log k); n extractions and at most n insertions give **O(n log k)** time with **O(k)** extra space (plus the output).

(c) The smallest element overall is within the first d + 1 positions, because it cannot be more than d positions right of position 1. In general, after the $i - 1$ smallest elements have been removed, the next smallest is among the first d + 1 remaining unread elements. Algorithm: insert A[1..d+1] into a min-heap. For i = 1 to n: extract-min, write it to output position i, and insert A[i + d + 1] if it exists. Each of the n extractions and insertions works on a heap of at most d + 1 elements, so **O(n log d)**. Correctness follows from the invariant that the heap always contains the smallest unoutput element: the element that belongs at position i sits at some original index in $[i - d, i + d]$, and all indices up to i + d have been inserted before the i-th extraction. This is the HW2 k-th-smallest and Tutorial 4 k-way-merge idea combined.